SwiftDrop — M2 Pricing & Zones

Design diagrams: what each one shows, where it lives, and what is drawn on it

1	Use case diagram	`usecase-m2-pricing.png`
2	Service integration diagram	`integration-m2-pricing.png`
3	Architecture / layering diagram	`architecture-m2-pricing-layers.png`
4	Data model (ER) diagram	`er-m2-pricing-zones.png`
5	Pricing rules class diagram	`class-m2-pricing-strategy.png`
6	Fee calculation activity diagram	`activity-m2-fee-calculation.png`
7	Rate-card selection activity diagram	`activity-m2-rate-card-selection.png`
8	Fee estimate sequence diagram	`sequence-m2-fee-estimate.png`
9	Quote lookup sequence diagram	`sequence-m2-quote-lookup.png`
10	Admin rate change sequence diagram	`sequence-m2-admin-rate-change.png`
11	Deployment diagram	`deployment-m2-kubernetes.png`

All images are PNG exports of the Mermaid sources in `doc/diagrams/m2-pricing-zones/`. Each diagram below has one page of detail followed by a full page of the image.

Use case diagram

Diagram name	`usecase-m2-pricing`
Image	`doc/diagrams/m2-pricing-zones/usecase-m2-pricing.png`
Source	`doc/diagrams/m2-pricing-zones/usecase-m2-pricing.mmd`
Vector export	`doc/diagrams/m2-pricing-zones/usecase-m2-pricing.svg`

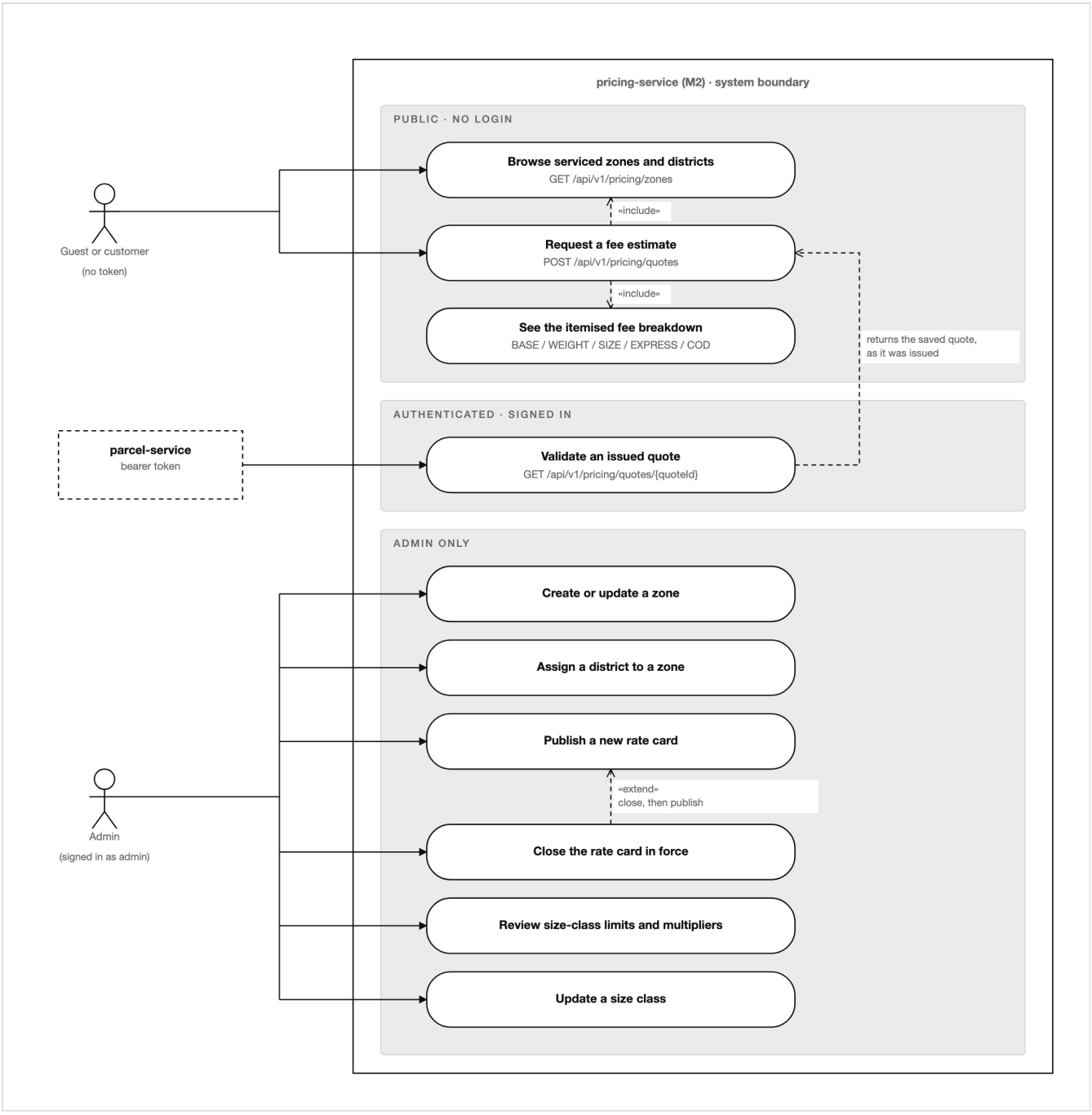
WHAT IT DEMONSTRATES

Who can use pricing-service and what each of them is allowed to do.

WHAT IS IN THE DIAGRAM

- Three actors: a guest or customer with no login, an admin, and parcel-service calling with a bearer token.
- Public band (no login): browse serviced zones and districts, request a fee estimate, and see the itemised fee breakdown of BASE, WEIGHT, SIZE, EXPRESS and COD lines.
- Signed-in band: validate an issued quote by its quote id.
- Admin band: create or update a zone, assign a district to a zone, publish a new rate card, close the rate card in force, review size-class limits and multipliers, and update a size class.
- The two «include» links show that a fee estimate always builds on the zone list and always produces a breakdown.
- The «extend» link shows that a price change is two steps: close the current card, then publish the new one.
- The dashed note records that a lookup returns the saved quote as it was issued.

Diagram 1 · Use case diagram · `usecase-m2-pricing.png`



Service integration diagram

Diagram name	`integration-m2-pricing`
Image	`doc/diagrams/m2-pricing-zones/integration-m2-pricing.png`
Source	`doc/diagrams/m2-pricing-zones/integration-m2-pricing.mmd`
Vector export	`doc/diagrams/m2-pricing-zones/integration-m2-pricing.svg`

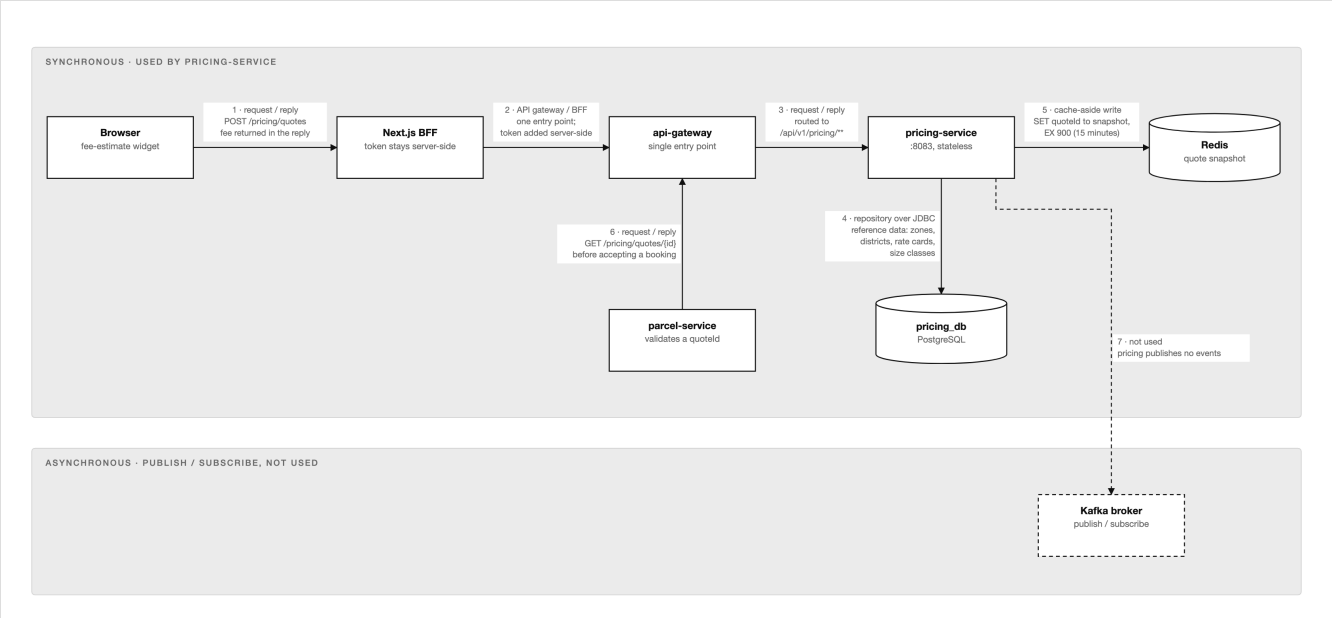
WHAT IT DEMONSTRATES

Which integration patterns pricing-service uses for a quote request, and which it does not use.

WHAT IS IN THE DIAGRAM

- Step 1: the browser widget calls the Next.js BFF with synchronous request/reply over REST.
- Step 2: the BFF forwards to the api-gateway, acting as the single entry point with the token added server-side.
- Step 3: the gateway routes the call to pricing-service under /api/v1/pricing/**.
- Step 4: pricing-service reads its reference data (zones, districts, rate cards, size classes) from Postgres through repositories.
- Step 5: the issued quote is written to Redis as a cache-aside snapshot, SET quoteId to snapshot with a 900 second expiry.
- Step 6: parcel-service calls back through the gateway to check a quote id before accepting a booking.
- Step 7, drawn dashed: the Kafka broker is not used, because pricing answers each request directly and publishes no events.

Diagram 2 · Service integration diagram · `integration-m2-pricing.png`



Architecture / layering diagram

Diagram name	`architecture-m2-pricing-layers`
Image	`doc/diagrams/m2-pricing-zones/architecture-m2-pricing-layers.png`
Source	`doc/diagrams/m2-pricing-zones/architecture-m2-pricing-layers.mmd`
Vector export	`doc/diagrams/m2-pricing-zones/architecture-m2-pricing-layers.svg`

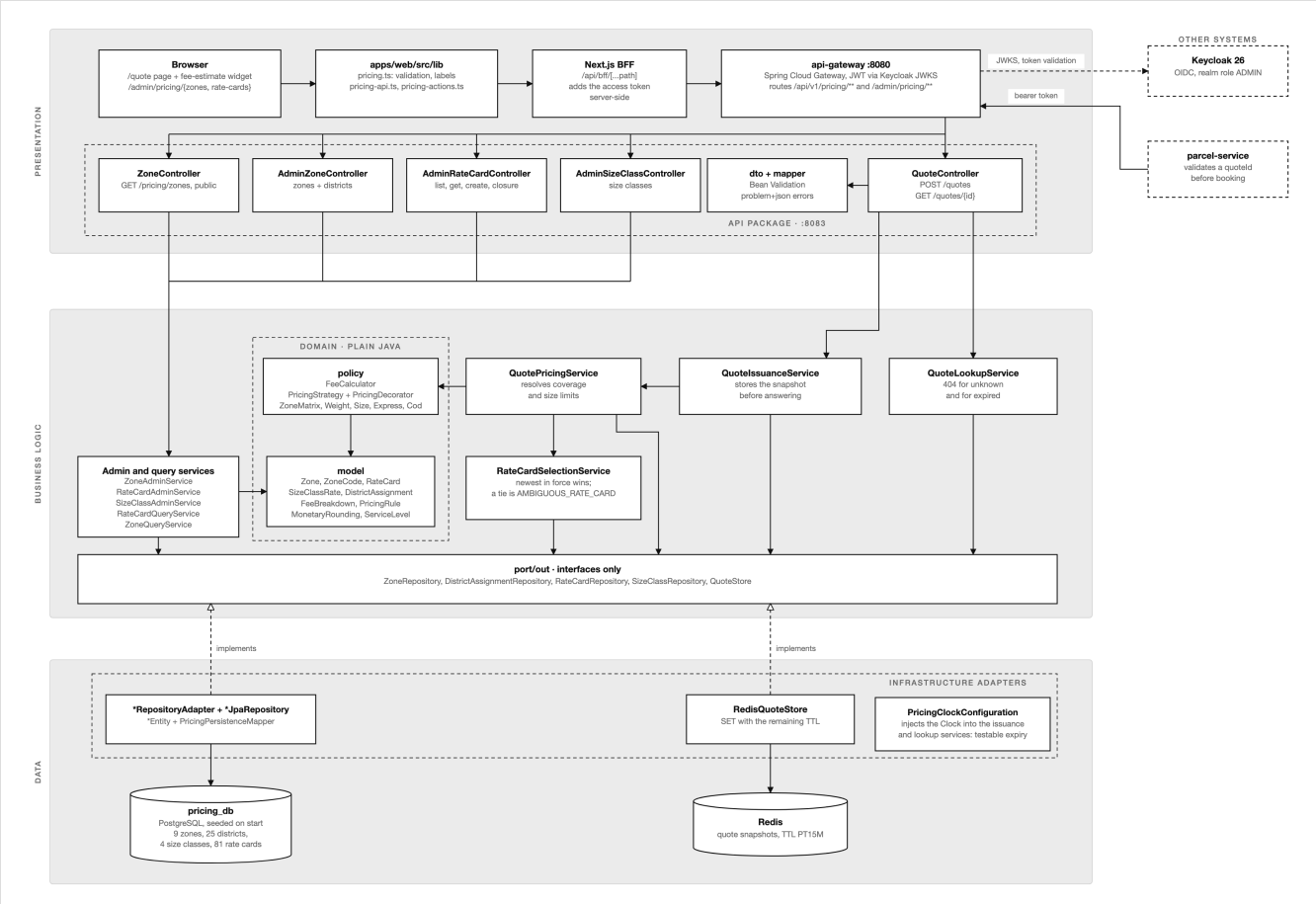
WHAT IT DEMONSTRATES

Where pricing-service sits in the system, and how it is layered inside.

WHAT IS IN THE DIAGRAM

- Presentation layer: the /quote page and fee-estimate widget, the admin pricing pages, the shared web library, the Next.js BFF, the api-gateway, and the five controllers with their DTOs and mappers.
- Business logic layer, application side: QuoteIssuanceService, QuotePricingService, RateCardSelectionService, QuoteLookupService, the admin and query services, and the outbound ports declared as interfaces only.
- Business logic layer, domain side: the policy package (FeeCalculator, strategies and decorators) and the model package (Zone, RateCard, SizeClassRate, FeeBreakdown, rounding and enums), written in plain Java with no framework types.
- Data layer: the JPA repository adapters and entities, RedisQuoteStore, the clock configuration, the Postgres pricing_db seeded with 9 zones, 25 districts, 4 size classes and 81 rate cards, and Redis holding quote snapshots with a 15-minute TTL.
- Dependencies point inwards only: api to application to domain, with infrastructure implementing the application's ports.
- Keycloak and parcel-service are shown outside the service as the systems it talks to.

Diagram 3 · Architecture / layering diagram · `architecture-m2-pricing-layers.png`



Data model (ER) diagram

Diagram name	`er-m2-pricing-zones`
Image	`doc/diagrams/m2-pricing-zones/er-m2-pricing-zones.png`
Source	`doc/diagrams/m2-pricing-zones/er-m2-pricing-zones.mmd`
Vector export	`doc/diagrams/m2-pricing-zones/er-m2-pricing-zones.svg`

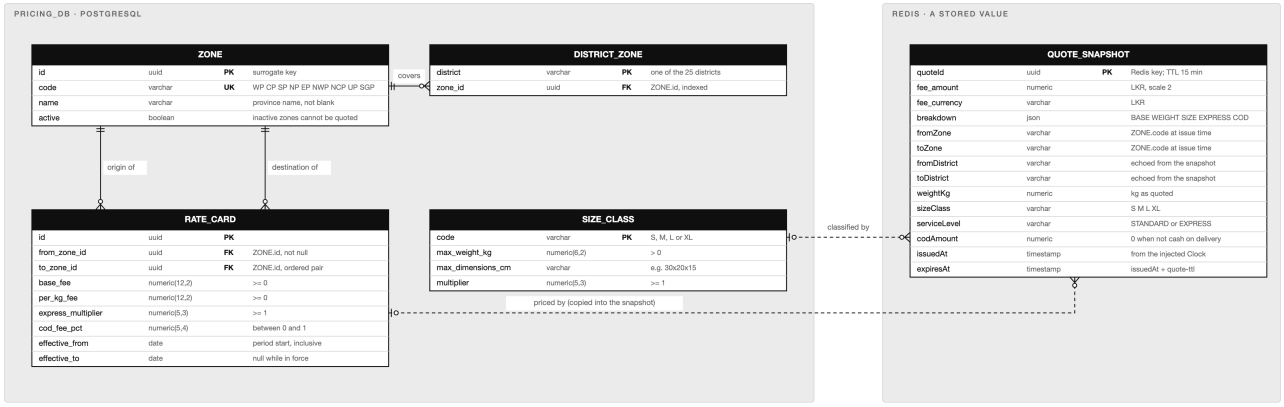
WHAT IT DEMONSTRATES

The pricing database tables with their real columns and constraints, and the quote snapshot that is kept in Redis rather than in a table.

WHAT IS IN THE DIAGRAM

- ZONE: surrogate uuid key, a unique code for each of the nine provinces, a name, and an active flag; an inactive zone cannot be quoted.
- DISTRICT_ZONE: each of the 25 districts maps to exactly one zone.
- SIZE_CLASS: S, M, L and XL with a maximum weight, maximum dimensions and a multiplier of at least 1.
- RATE_CARD: an ordered zone pair with base fee, per-kg fee, express multiplier and COD percentage, valid from a start date until an end date that is empty while the card is in force.
- QUOTE_SNAPSHOT: stored in Redis under an opaque uuid with a 15-minute TTL, holding the fee, the ordered breakdown, both zones and districts, weight, size class, service level, COD amount, and the issue and expiry times.
- Crow's-foot lines are foreign keys inside the database; dashed lines mark values copied into the snapshot, so closing a rate card leaves quotes already issued unchanged.

Diagram 4 · Data model (ER) diagram · `er-m2-pricing-zones.png`



Solid crow's-foot line: foreign key in pricing_db. Dashed line: the value is copied into the snapshot, so closing a rate card leaves issued quotes unchanged.
QUOTE_SNAPSHOT is stored in Redis under an opaque UUID with a 15-minute TTL (swiftdrop.pricing.quote-ttl) and returned verbatim.

Pricing rules class diagram

Diagram name	`class-m2-pricing-strategy`
Image	`doc/diagrams/m2-pricing-zones/class-m2-pricing-strategy.png`
Source	`doc/diagrams/m2-pricing-zones/class-m2-pricing-strategy.mmd`
Vector export	`doc/diagrams/m2-pricing-zones/class-m2-pricing-strategy.svg`

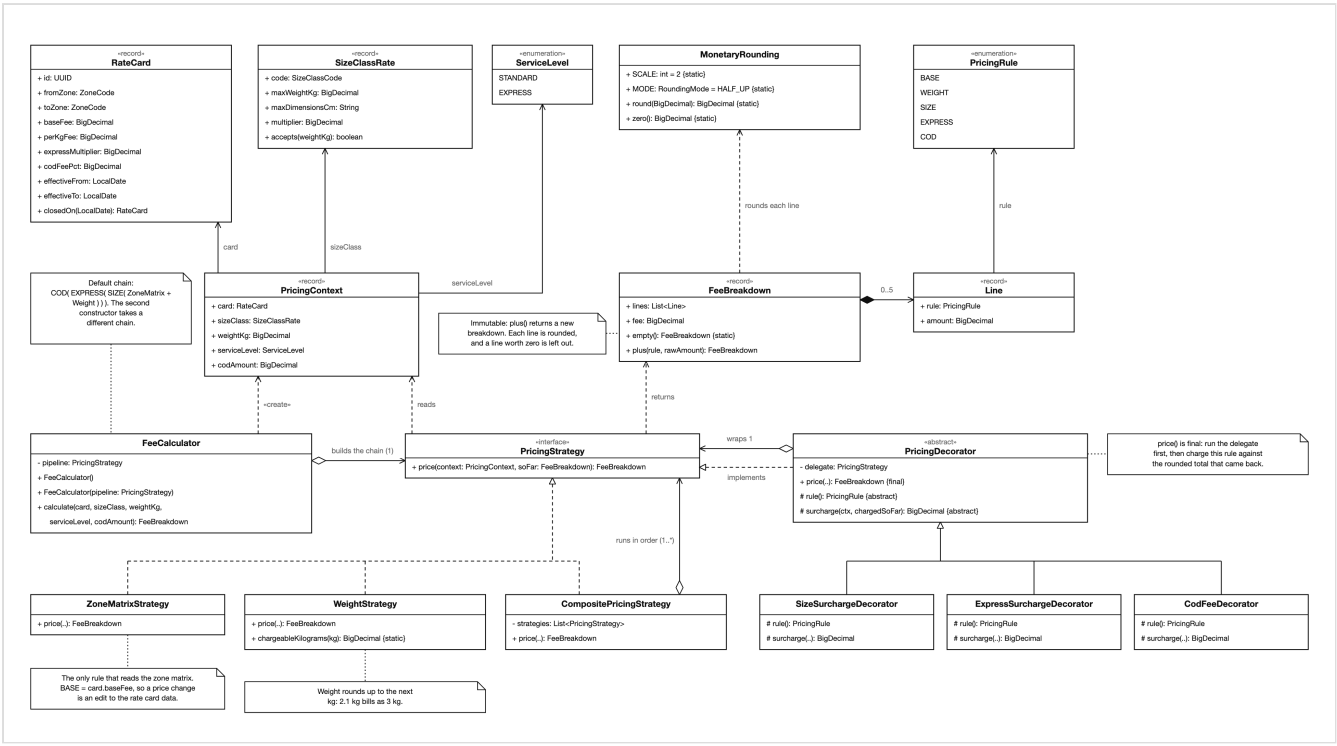
WHAT IT DEMONSTRATES

How the fee rules are arranged as classes, using the Strategy and Decorator patterns.

WHAT IS IN THE DIAGRAM

- PricingStrategy is the interface every rule implements: it takes the pricing context and the breakdown so far, and returns a new breakdown.
- ZoneMatrixStrategy reads the rate card and contributes the BASE line; WeightStrategy charges the per-kg fee after rounding the weight up to the next whole kilogram.
- CompositePricingStrategy runs a list of strategies in order.
- PricingDecorator is the abstract wrapper whose price() method is final: it runs the strategy it wraps, then charges its own rule against the rounded total.
- SizeSurchargeDecorator, ExpressSurchargeDecorator and CodFeeDecorator are the three concrete surcharges.
- FeeCalculator assembles the default chain COD(EXPRESS(SIZE(ZoneMatrix + Weight))); a second constructor accepts a different chain.
- Supporting types: the PricingContext and FeeBreakdown records, the Line record, the PricingRule and ServiceLevel enumerations, MonetaryRounding at 2 decimals HALF_UP, and the RateCard and SizeClassRate records.
- Relationships use UML notation: realization for the interface, generalization for the decorators, aggregation for what a class wraps or assembles, and composition for the lines inside a breakdown.

Diagram 5 · Pricing rules class diagram · `class-m2-pricing-strategy.png`



Fee calculation activity diagram

Diagram name	`activity-m2-fee-calculation`
Image	`doc/diagrams/m2-pricing-zones/activity-m2-fee-calculation.png`
Source	`doc/diagrams/m2-pricing-zones/activity-m2-fee-calculation.mmd`
Vector export	`doc/diagrams/m2-pricing-zones/activity-m2-fee-calculation.svg`

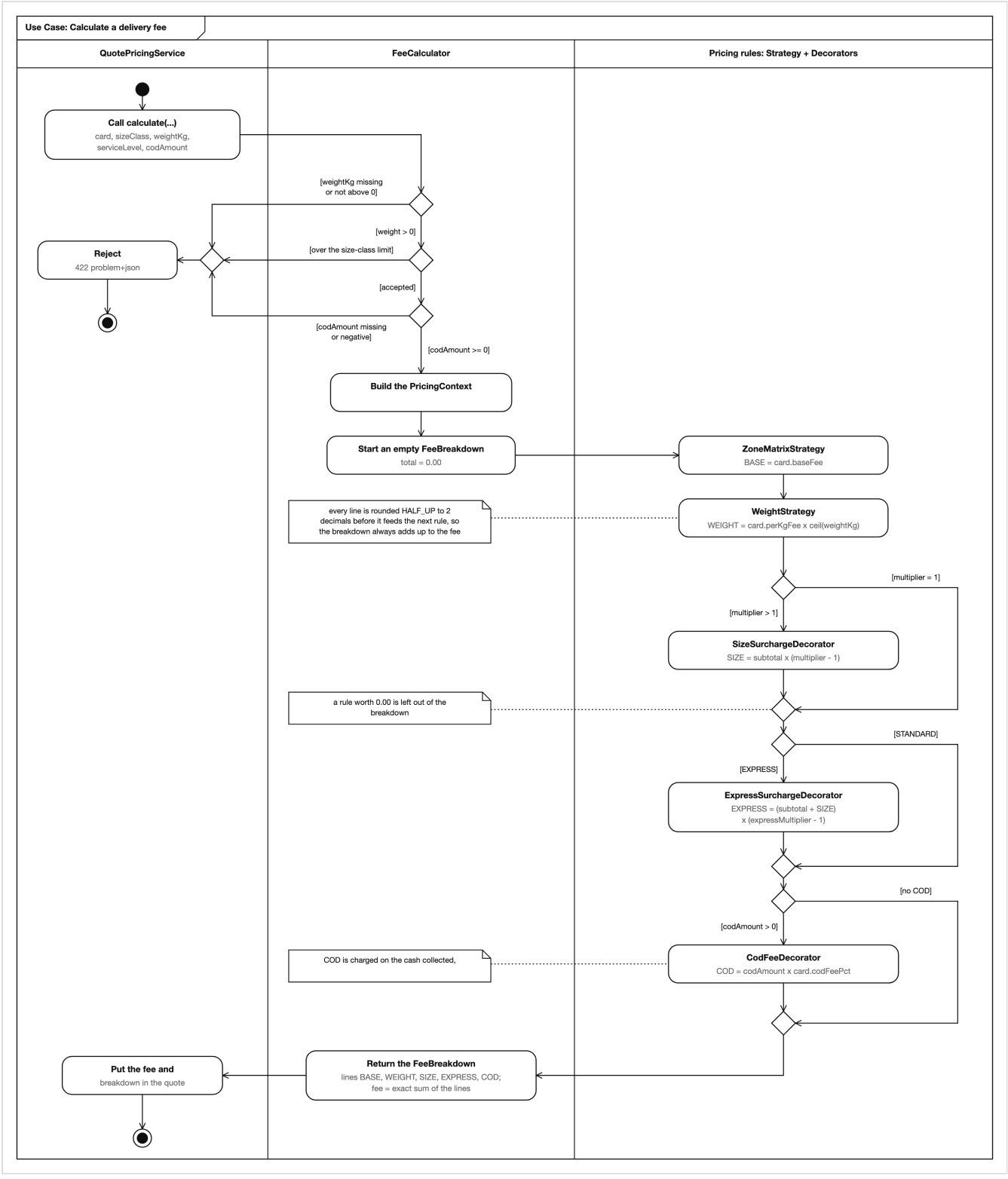
WHAT IT DEMONSTRATES

Step by step, how one fee is worked out and what ends up in the breakdown.

WHAT IS IN THE DIAGRAM

- Three input guards run first: the weight must be present and above zero, the size class must accept that weight, and the COD amount must not be negative. Any failure is rejected as 422.
- An empty breakdown is created, with a running total of 0.00.
- BASE comes straight from the rate card's base fee.
- WEIGHT is the per-kg fee times the weight rounded up to the next whole kilogram.
- SIZE is charged only when the size class multiplier is above 1, applied to the subtotal so far.
- EXPRESS is charged only for express service, applied to the subtotal including the size surcharge.
- COD is charged only when there is cash to collect, as a percentage of the cash amount.
- Each line is rounded to 2 decimals before the next rule runs, and a line that works out to 0.00 is left out, so the lines shown always add up to the total charged.

Diagram 6 · Fee calculation activity diagram · `activity-m2-fee-calculation.png`



Rate-card selection activity diagram

Diagram name	`activity-m2-rate-card-selection`
Image	`doc/diagrams/m2-pricing-zones/activity-m2-rate-card-selection.png`
Source	`doc/diagrams/m2-pricing-zones/activity-m2-rate-card-selection.mmd`
Vector export	`doc/diagrams/m2-pricing-zones/activity-m2-rate-card-selection.svg`

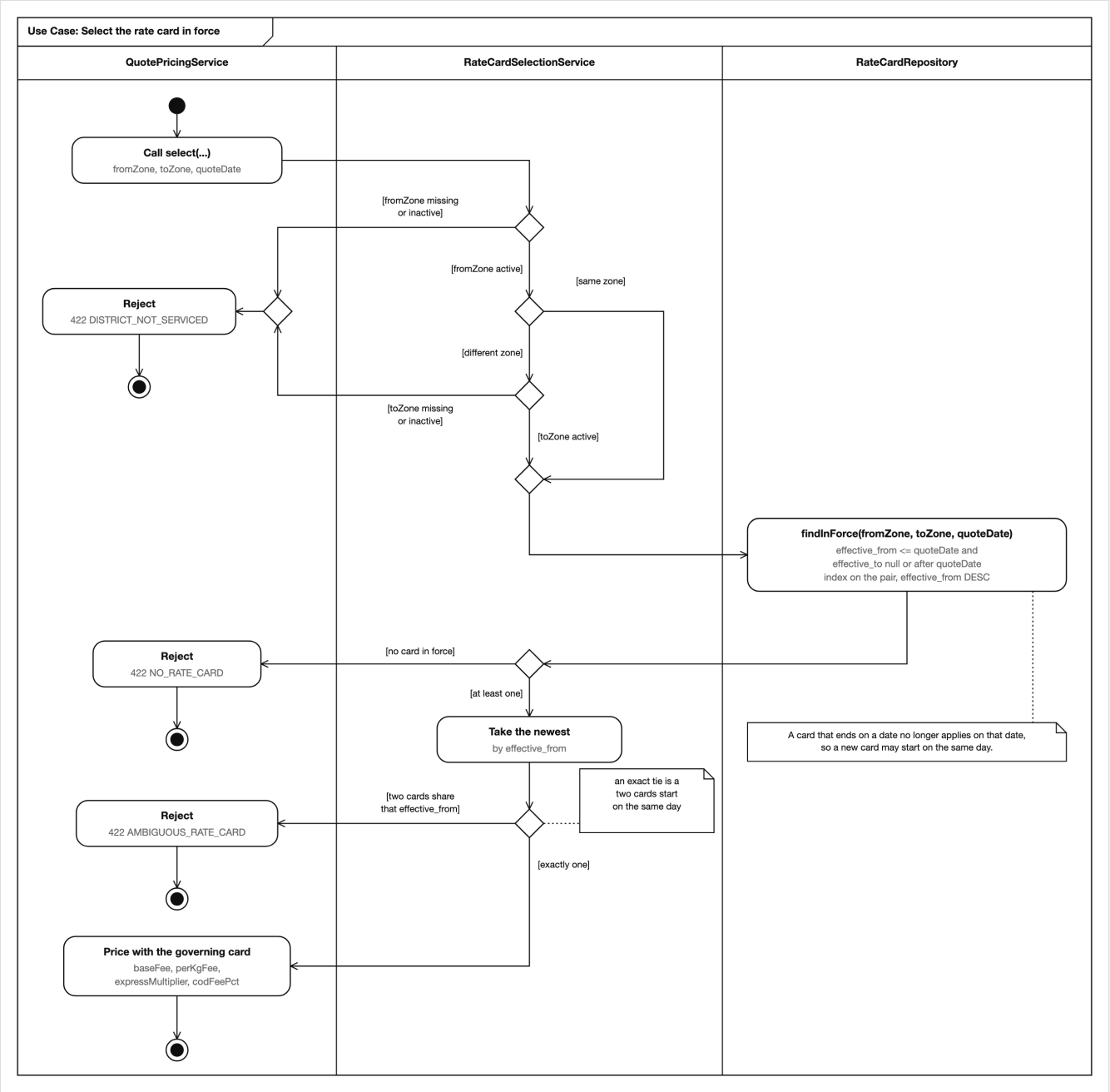
WHAT IT DEMONSTRATES

How the service decides which rate card applies to a delivery on a given date.

WHAT IS IN THE DIAGRAM

- Both zones must exist and be active; otherwise the answer is 422 DISTRICT_NOT_SERVICED.
- Deliveries inside one zone are allowed and use the same-zone card.
- Cards in force are those whose start date is on or before the quote date and whose end date is empty or later, newest start date first.
- If no card is in force the answer is 422 NO_RATE_CARD.
- The card with the newest start date wins.
- If two cards share that start date the answer is 422 AMBIGUOUS_RATE_CARD, because that is a setup mistake rather than something to guess at.
- The result is the governing card: base fee, per-kg fee, express multiplier and COD percentage.

Diagram 7 · Rate-card selection activity diagram · `activity-m2-rate-card-selection.png`



Fee estimate sequence diagram

Diagram name	`sequence-m2-fee-estimate`
Image	`doc/diagrams/m2-pricing-zones/sequence-m2-fee-estimate.png`
Source	`doc/diagrams/m2-pricing-zones/sequence-m2-fee-estimate.mmd`
Vector export	`doc/diagrams/m2-pricing-zones/sequence-m2-fee-estimate.svg`

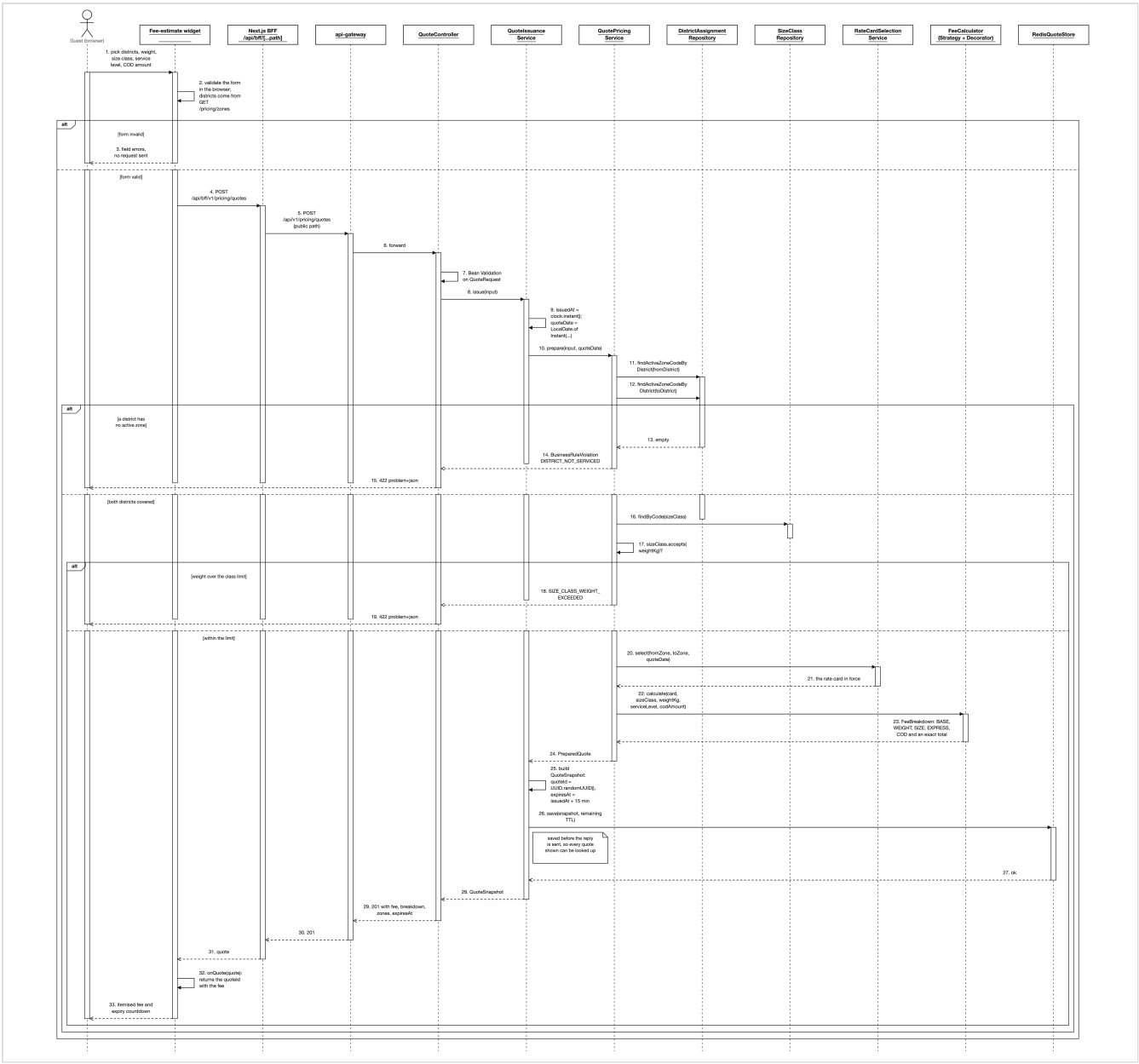
WHAT IT DEMONSTRATES

The full path of an anonymous fee estimate, from the browser form to the stored quote and back.

WHAT IS IN THE DIAGRAM

- The widget validates the form in the browser first, with districts taken from the public zones endpoint; invalid input never leaves the page.
- The request goes through the BFF and the gateway as a public path, with no token needed, and is validated again at the controller.
- QuoteIssuanceService stamps the issue time from the clock and passes the quote date on.
- QuotePricingService resolves both districts to zones, loads the size class and checks the weight against it.
- RateCardSelectionService returns the card in force, and FeeCalculator produces the itemised breakdown.
- A snapshot is built with a fresh quote id and an expiry 15 minutes after the issue time, and is saved to Redis before the reply is sent, so every quote shown can be looked up.
- The response is 201 with the fee, the breakdown, the zones and the expiry time; the widget shows the itemised fee and a countdown.
- Alternative branches cover an invalid form, a district with no active zone (422 DISTRICT_NOT_SERVICED) and a weight over the size-class limit (422 SIZE_CLASS_WEIGHT_EXCEEDED).

Diagram 8 · Fee estimate sequence diagram · `sequence-m2-fee-estimate.png`



Quote lookup sequence diagram

Diagram name	`sequence-m2-quote-lookup`
Image	`doc/diagrams/m2-pricing-zones/sequence-m2-quote-lookup.png`
Source	`doc/diagrams/m2-pricing-zones/sequence-m2-quote-lookup.mmd`
Vector export	`doc/diagrams/m2-pricing-zones/sequence-m2-quote-lookup.svg`

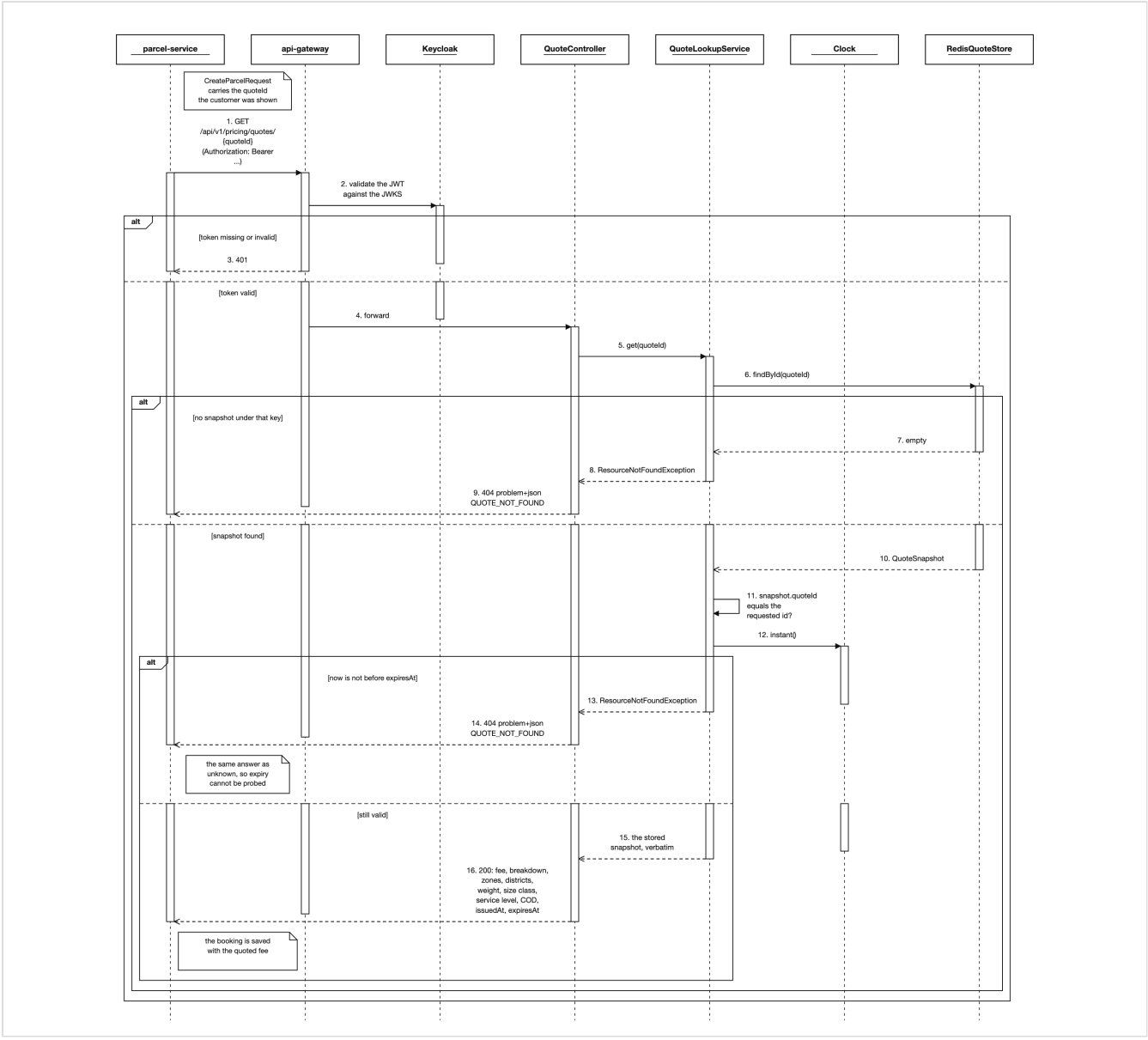
WHAT IT DEMONSTRATES

How another service checks a quote it was given, before relying on that price.

WHAT IS IN THE DIAGRAM

- The caller sends the quote id with a bearer token; the gateway validates the token against Keycloak's JWKS and answers 401 if it is missing or invalid.
- QuoteLookupService reads the snapshot from Redis by quote id.
- If nothing is stored under that key the answer is 404 QUOTE_NOT_FOUND.
- The injected clock is compared with the stored expiry time.
- An expired quote gets exactly the same 404 as an unknown one, so expiry cannot be probed by guessing ids.
- A valid quote is returned as stored: fee, breakdown, zones, districts, weight, size class, service level, COD amount, issue and expiry times.
- The booking is then saved with the fee the customer was quoted.

Diagram 9 · Quote lookup sequence diagram · `sequence-m2-quote-lookup.png`



Admin rate change sequence diagram

Diagram name	`sequence-m2-admin-rate-change`
Image	`doc/diagrams/m2-pricing-zones/sequence-m2-admin-rate-change.png`
Source	`doc/diagrams/m2-pricing-zones/sequence-m2-admin-rate-change.mmd`
Vector export	`doc/diagrams/m2-pricing-zones/sequence-m2-admin-rate-change.svg`

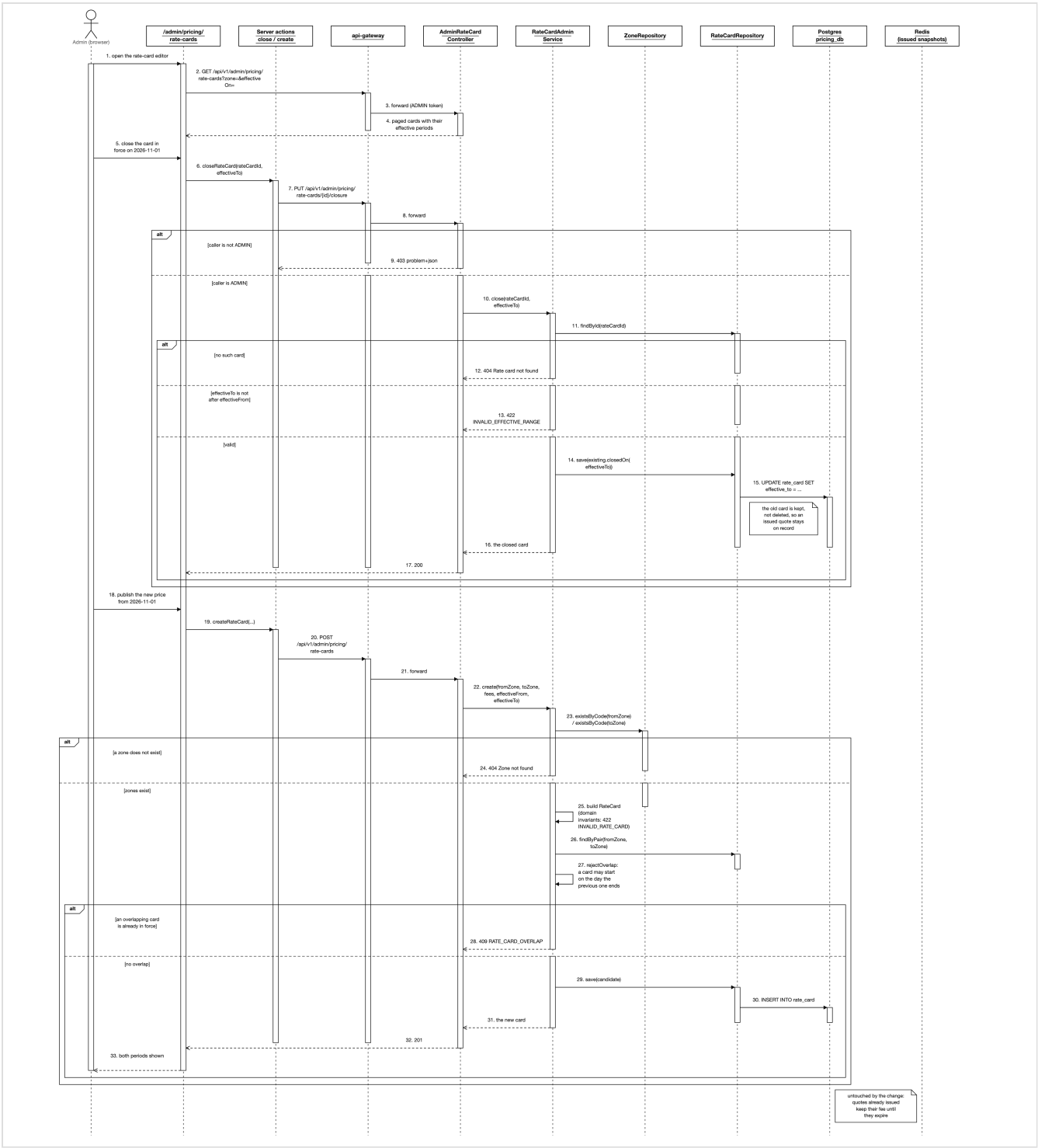
WHAT IT DEMONSTRATES

How an admin changes the price of a zone-to-zone lane without disturbing quotes already issued.

WHAT IS IN THE DIAGRAM

- The admin opens the rate-card editor and the UI lists the cards with their effective periods.
- Step one: close the card in force by setting an end date through the closure endpoint.
- Step two: publish the replacement card starting on that same date.
- Only an admin token is accepted; any other caller gets 403.
- Validation branches: 404 for an unknown card or an unknown zone, 422 for an end date that is not after the start date or a card that breaks its own invariants, and 409 when the new period overlaps an existing one.
- A card may start on the day the previous one ends, so the two periods leave neither gap nor overlap.
- The closure is an update, not a delete, so the old card stays on record.
- The closing note records that Redis snapshots are untouched, so quotes already issued keep their fee until they expire.

Diagram 10 · Admin rate change sequence diagram · `sequence-m2-admin-rate-change.png`



Deployment diagram

Diagram name	`deployment-m2-kubernetes`
Image	`doc/diagrams/m2-pricing-zones/deployment-m2-kubernetes.png`
Source	`doc/diagrams/m2-pricing-zones/deployment-m2-kubernetes.mmd`
Vector export	`doc/diagrams/m2-pricing-zones/deployment-m2-kubernetes.svg`

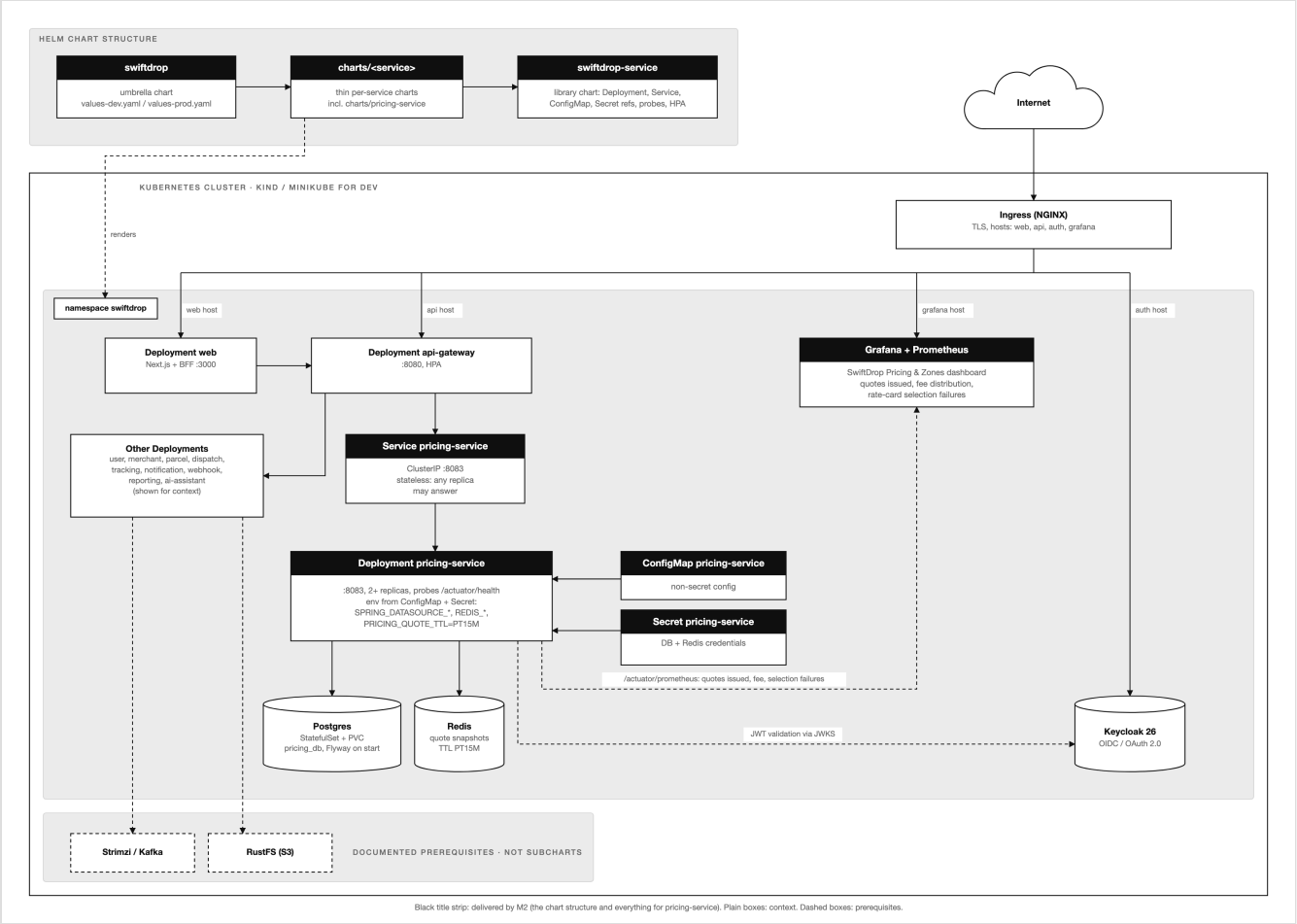
WHAT IT DEMONSTRATES

How the services are packaged with Helm and what runs where in the Kubernetes cluster.

WHAT IS IN THE DIAGRAM

- Chart structure: one shared library chart holding the Deployment, Service, ConfigMap, Secret references, probes and optional autoscaling; one thin chart per service; and one umbrella chart with dev and prod values.
- Traffic enters through an NGINX ingress with separate hosts for the web, api, auth and grafana endpoints.
- The namespace runs the Next.js web deployment, the api-gateway, and pricing-service with two or more replicas, health probes, and configuration from a ConfigMap and a Secret including the 15-minute quote TTL.
- The pricing-service ClusterIP service is stateless, so any replica may answer.
- The other service deployments are drawn for context only.
- Postgres runs as a StatefulSet with a volume and the pricing database, alongside Redis for snapshots and Keycloak for tokens, all templated by the umbrella chart so the images match the compose file.
- Grafana and Prometheus carry the pricing dashboard, fed from the service's metrics endpoint.
- Strimzi and RustFS are listed as prerequisites rather than charts.

Diagram 11 · Deployment diagram · `deployment-m2-kubernetes.png`



Black title strip: delivered by M2 (the chart structure and everything for pricing-service). Plain boxes: context. Dashed boxes: prerequisites.